Лекция №1

Информационные технологии, среда разработки и контроль версий

Дисциплина: «Информационные технологии и программирование»

Содержание

1. Роль информационных технологий в современной инженерии
2. Операционная система и файловая модель
3. Командная строка как инструмент разработчика
4. Среда Visual Studio Code
5. Принципы контроля версий
6. Язык разметки Markdown

1. Роль информационных технологий в современной инженерии

Автоматизация и цифровые процессы

Цель внедрения ИТ — сократить ручной труд, уменьшить число ошибок, ускорить разработку.

Пример автоматизированного конвейера инженера:

1. подготовить исходные данные в таблице;
2. выполнить расчёт скриптом на Python;
3. автоматически сформировать отчёт;
4. сохранить результаты в репозитории;
5. построить графики и диаграммы.

Ручной вариант против автоматизированного

Процесс	Ручной вариант	Автоматизированный вариант
Создание отчёта	Текст вручную	Генерация из данных скриптом
Расчёт параметров	Калькулятор	Python / MATLAB
Хранение версий чертежей	Папки с копиями	Git-репозиторий
Проверка данных	Просмотр глазами	Скрипт валидации
Публикация документации	Передача файла	Сборка через CI/CD

Программные инструменты в инженерии

Категория	Назначение	Примеры
CAD-системы	Чертежи и 3D-модели	AutoCAD, SolidWorks, КОМПАС-3D
CAE-системы	Расчёты и моделирование	ANSYS, Simulink
IDE и редакторы	Разработка кода	VS Code, PyCharm
Системы контроля версий	История изменений	Git, GitHub, GitLab
CI/CD-системы	Автосборка и проверка	GitLab CI, GitHub Actions
Средства документирования	Техническая документация	Markdown, Typst, LaTeX

Культура разработки

- **Контроль версий** — все изменения фиксируются в репозитории.
- **Code Review** — проверка изменений коллегами перед слиянием.
- **Документация как код** — документация хранится в проекте как текст.
- **Автотесты** — проверка корректности модулей.
- **Воспроизводимость** — расчёт можно повторить на другом компьютере.

> Чем позже найдена ошибка, тем дороже исправление:
> $C_{prod} \approx 10 \dots 100 \cdot C_{dev}$

Культура разработки: сообщения коммитов

✅ Хорошо — команде понятно, что сделано:

```
git commit -m "feat: add temperature sensor data parser"
```

❌ Плохо — сообщение ничего не объясняет:

```
git commit -m "fix"
```

? Вопросы по разделу 1

1. Какова основная цель автоматизации инженерной деятельности?
2. Какие классы программных инструментов вы знаете? Приведите примеры.
3. Какие практики входят в современную культуру разработки?
4. Почему стоимость исправления ошибки растёт по мере позднего обнаружения?

2. Операционная система и файловая модель

Основные понятия файловой системы

- **Файл** — именованная последовательность данных на носителе.
- **Каталог (директория)** — объект, содержащий файлы и другие каталоги.
- **inode** — дескриптор метаданных файла: размер, права, владелец, время; имя файла хранится в каталоге.

Примеры файлов проекта:

```
main.py   report.md   config.json   data.csv
```

Структура проекта и пути доступа

```
project/
├── README.md
├── data/
│   └── input.csv
├── src/
│   └── main.py
└── tests/
    └── test_main.py
```

- **Абсолютный путь** — от корня: `/home/student/project/main.py`
- **Относительный путь** — от текущего каталога: `src/main.py`, `../config.json`
- Обозначения: `.` — текущий каталог, `..` — родительский, `~` — домашний

Права доступа в Unix-подобных системах

Три категории: **владелец**, **группа**, **остальные**.

Права: чтение `r` = 4, запись `w` = 2, выполнение `x` = 1.

```
rwx r-x r--   →   7 5 4
```

Права	Значение	Применение
`644`	владелец: чтение/запись; остальные: чтение	обычный файл
`755`	владелец: всё; остальные: чтение/выполнение	скрипт
`700`	доступ только у владельца	приватный каталог
`600`	чтение/запись только у владельца	ключи, секреты

Просмотр и изменение прав

```
ls -l
-rw-r--r-- 1 student student  120 Jan 23 15:42 README.md
-rwxr-xr-x 1 student student  340 Jan 23 15:45 script.sh
drwxr-xr-x 2 student student 4096 Jan 23 15:40 src/
```

Первый символ: `-` — файл, `d` — каталог; далее три группы `rwx`.

```
chmod 755 script.sh   # сделать скрипт исполняемым
chmod 600 config.json # закрыть доступ другим пользователям
```

? Вопросы по разделу 2

1. Чем файл отличается от каталога? Что хранит inode?
2. Чем абсолютный путь отличается от относительного? Приведите примеры.
3. Что означают права `rwxr-xr--`? Каков их числовой код?
4. Какой командой изменить права доступа? Приведите пример.

3. Командная строка как инструмент разработчика

Запуск терминала и навигация

- Linux: `Ctrl + Alt + T`; macOS: `Cmd + Space → Terminal`; VS Code: `Ctrl + ``

Команда	Действие
`pwd`	показать полный абсолютный путь
`cd projects`	перейти в каталог
`cd ..`	подняться на уровень выше
`cd ~`	в домашний каталог
`ls -la`	список файлов, включая скрытые

Создание и управление файлами

```
touch main.py                  # создать пустой файл
mkdir data                     # создать каталог
mkdir -p project/src/utils     # создать вложенную структуру
cp main.py backup.py           # копировать файл
cp -r src src_backup           # копировать каталог
mv old.py new.py               # переименовать / переместить
rm -i file.txt                 # удалить с подтверждением
```

> Команда `rm` удаляет файлы безвозвратно — работайте осторожно!

Просмотр, перенаправление, конвейеры

```
cat README.md         # вывести файл целиком
head -n 5 data.csv    # первые 5 строк
tail -n 20 data.csv   # последние 20 строк

echo "Hello" > hello.txt   # создать / перезаписать файл
echo "New"   >> hello.txt  # добавить в конец файла

ls -la | grep ".py"        # конвейер: вывод ls → вход grep
grep -r "TODO" .           # поиск текста во всех файлах
find . -name "*.py"        # поиск файлов по имени
```

? Вопросы по разделу 3

1. Какие команды используются для навигации по каталогам?
2. Как одной командой создать вложенную структуру каталогов?
3. В чём разница между `>` и `>>` при перенаправлении вывода?
4. Что такое конвейер? Приведите пример.

4. Среда Visual Studio Code

Интерфейс VS Code

- **Activity Bar** — вертикальная панель разделов слева.
- **Side Bar** — проводник, поиск, Git, расширения.
- **Editor Group** — область редактирования файлов.
- **Panel** — терминал, проблемы, вывод, отладка.
- **Status Bar** — строка состояния внизу окна.

Раздел	Назначение
Explorer	файлы проекта
Search	поиск по проекту
Source Control	работа с Git
Extensions	установка расширений

Команды и горячие клавиши

Действие	Сочетание
Палитра команд	`Ctrl + Shift + P`
Быстрое открытие файла	`Ctrl + P`
Терминал	`Ctrl + ` `
Сохранить файл	`Ctrl + S`
Поиск по проекту	`Ctrl + Shift + F`
Закомментировать строку	`Ctrl + /`

Открытие проекта: `File → Open Folder` или команда `code .`

Расширения для учебного курса

Расширение	Назначение
Python / Pylance	поддержка и анализ кода Python
GitLens	расширенная работа с Git
Markdown All in One	поддержка Markdown
markdownlint	проверка стиля Markdown
Live Server	локальный сервер для HTML/CSS/JS
Remote - SSH	подключение к удалённым серверам

Установка: `Ctrl + Shift + X` → поиск → **Install**.

Настройки: `settings.json`

```
{
  "editor.fontSize": 15,
  "editor.tabSize": 4,
  "editor.formatOnSave": true,
  "files.autoSave": "onFocusChange",
  "files.exclude": { "**/__pycache__": true },
  "python.defaultInterpreterPath": "/usr/bin/python3"
}
```

- **User-настройки** — действуют для всех проектов пользователя.
- **Workspace-настройки** (`.vscode/settings.json`) — только для текущего проекта.

Встроенный терминал и запуск Python

```
python3 --version
mkdir examples && cd examples
touch hello.py
python3 hello.py
```

```
name = input("Введите имя: ")
print(f"Привет, {name}!")
```

Выбор интерпретатора: `Ctrl + Shift + P → Python: Select Interpreter`

? Вопросы по разделу 4

1. Из каких основных элементов состоит окно VS Code?
2. Как открыть палитру команд? Приведите примеры полезных команд.
3. Чем пользовательские настройки отличаются от настроек рабочего пространства?
4. Какие расширения полезны для учебного курса и зачем?
5. Как открыть встроенный терминал и запустить Python-скрипт?

5. Принципы контроля версий

Зачем нужен контроль версий

Плохой способ хранения версий:

```
project_final/  project_final_v2/
project_final_new/  project_final_real_final/
```

Хороший способ — Git:

```
git init
git add .
git commit -m "feat: add initial version"
```

Git хранит **историю изменений**, позволяет откатываться, работать в команде и вести ветки.

Основные области Git

```
Working Directory --git add--> Staging Area
Staging Area --git commit--> Local Repository
Local Repository --git push--> Remote Repository
Remote Repository --git pull--> Working Directory
```

Область	Назначение	Команда просмотра
Working Directory	файлы на диске	`git status`
Staging Area	подготовленные изменения	`git diff --staged`
Local Repository	история коммитов	`git log --oneline`

Базовые команды Git

```
mkdir my-project && cd my-project
git init                          # создать репозиторий
echo "# Мой проект" > README.md
git status                        # посмотреть состояние
git add README.md                 # добавить файл в индекс
git commit -m "docs: add README"  # зафиксировать изменения
git log --oneline --graph --decorate  # история коммитов
```

Коммит — снимок проекта с уникальным хешем, автором, датой и сообщением.

Ветки и слияние

```
git switch -c feature/readme   # создать ветку и перейти в неё
echo "## Описание" >> README.md
git add README.md
git commit -m "docs: add description"
git switch main                # вернуться в основную ветку
git merge feature/readme       # слить изменения
```

Ветка — указатель на коммит: изменения разрабатываются изолированно и затем объединяются.

Удалённые репозитории и отмена изменений

```
git clone https://github.com/user/project.git  # клонировать
git push origin main     # отправить изменения
git pull origin main     # получить изменения
```

Команда	Действие	Безопасность
`git restore file.txt`	отменить изменения файла	высокая
`git revert HEAD`	новый коммит, отменяющий предыдущий	высокая
`git reset --hard HEAD~1`	удалить коммит и изменения	низкая

? Вопросы по разделу 5

1. Какие задачи решает контроль версий?
2. Назовите основные области Git и команды перехода между ними.
3. Что такое ветка и зачем она нужна?
4. Чем `git revert` отличается от `git reset --hard`?
5. Какие команды используются для работы с удалённым репозиторием?

6. Язык разметки Markdown

Зачем нужен Markdown

- исходный текст читается без специальных программ;
- файлы хранятся как обычный текст и дружат с Git;
- поддерживается GitHub, GitLab, VS Code, Jupyter;
- конвертируется в HTML, PDF и другие форматы.

```
# Заголовок 1
## Заголовок 2
**жирный**, *курсив*, ~~зачёркнутый~~, `код в строке`
```

Списки, ссылки, изображения, код

```
- пункт списка              [текст ссылки](https://example.com)
1. первый шаг               ![альт. текст](images/scheme.png)
   - вложенный пункт

> цитата

Список задач: `- [x] сделано`, `- [ ] предстоит`


# Таблицы

| Команда     | Назначение              |
|-------------|-------------------------|
| `git init`  | создать репозиторий     |
| `git add`   | добавить в индекс       |
```

Типичные ошибки

Ошибка	Правильно
`#Заголовок` без пробела	`# Заголовок`
нет пустой строки перед списком/таблицей	добавлять пустую строку
незакрытый блок кода	закрывать тройными кавычками
нарушение иерархии заголовков	соблюдать H1 → H2 → H3

? Вопросы по разделу 6

1. Какие преимущества Markdown перед другими форматами документации?
2. Как создать заголовки, списки и выделение текста?
3. Как вставить код, ссылку и изображение?
4. Какие частые ошибки встречаются при оформлении Markdown?

Итоги лекции

1. ИТ — неотъемлемая часть современной инженерии.
2. Необходимо понимать файловую систему, пути и права доступа.
3. Командная строка — базовый инструмент разработчика.
4. VS Code — гибкая среда разработки с расширениями и терминалом.
5. Git хранит историю изменений и позволяет безопасно откатываться.
6. Markdown — стандарт оформления документации и `README.md`.